tf.compat.v1.nn.dynamic_rnn Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/dynamic_rnn

> *Creates a recurrent neural network specified by RNNCell cell. (deprecated)*

Function Signatures

```
tf.compat.v1.nn.dynamic_rnn( cell, inputs,
sequence_length=None, initial_state=None, dtype=None,
parallel_iterations=None, swap_memory=False,
time_major=False, scope=None )
```

Code Examples

```
tf.compat.v1.nn.dynamic_rnn(
cell,
inputs,
sequence_length=None,
initial_state=None,
dtype=None,
parallel_iterations=None,
swap_memory=False,
time_major=False,
scope=None
)
```

```
tf.compat.v1.nn.dynamic_rnn(
cell,
inputs,
sequence_length=None,
initial_state=None,
dtype=None,
parallel_iterations=None,
swap_memory=False,
time_major=False,
scope=None
)
```

```
# create 2 LSTMCells
rnn_layers = [tf.compat.v1.nn.rnn_cell.LSTMCell(size) for size in
[128, 256]]
# create a RNN cell composed sequentially of a number of RNNCells
multi_rnn_cell = tf.compat.v1.nn.rnn_cell.MultiRNNCell(rnn_layers)
# 'outputs' is a tensor of shape [batch_size, max_time, 256]
# 'state' is a N-tuple where N is the number of LSTMCells
containing a
# tf.nn.rnn_cell.LSTMStateTuple for each cell
outputs, state = tf.compat.v1.nn.dynamic_rnn(cell=multi_rnn_cell,
inputs=data,
dtype=tf.float32)
```

```
# create 2 LSTMCells
rnn_layers = [tf.compat.v1.nn.rnn_cell.LSTMCell(size) for size in
[128, 256]]
# create a RNN cell composed sequentially of a number of RNNCells
multi_rnn_cell = tf.compat.v1.nn.rnn_cell.MultiRNNCell(rnn_layers)
# 'outputs' is a tensor of shape [batch_size, max_time, 256]
# 'state' is a N-tuple where N is the number of LSTMCells
containing a
# tf.nn.rnn_cell.LSTMStateTuple for each cell
outputs, state = tf.compat.v1.nn.dynamic_rnn(cell=multi_rnn_cell,
inputs=data,
dtype=tf.float32)
```

```
# RNN layer can take a list of cells, which will then stack them
together.
# By default, keras RNN will only return the last timestep output
and will not
# return states. If you need whole time sequence output as well as
the states,
# you can set `return_sequences` and `return_state` to True.
rnn_layer = tf.keras.layers.RNN([tf.keras.layers.LSTMCell(128),
tf.keras.layers.LSTMCell(256)],
return_sequences=True,
return_state=True)
outputs, output_states = rnn_layer(inputs, states)
```


```
# RNN layer can take a list of cells, which will then stack them
together.
# By default, keras RNN will only return the last timestep output
and will not
# return states. If you need whole time sequence output as well as
the states,
# you can set `return_sequences` and `return_state` to True.
rnn_layer = tf.keras.layers.RNN([tf.keras.layers.LSTMCell(128),
tf.keras.layers.LSTMCell(256)],
return_sequences=True,
return_state=True)
outputs, output_states = rnn_layer(inputs, states)
```

```
# create a BasicRNNCell
rnn_cell = tf.compat.v1.nn.rnn_cell.BasicRNNCell(hidden_size)
# 'outputs' is a tensor of shape [batch_size, max_time,
cell_state_size]
# defining initial state
initial_state = rnn_cell.zero_state(batch_size, dtype=tf.float32)
# 'state' is a tensor of shape [batch_size, cell_state_size]
outputs, state = tf.compat.v1.nn.dynamic_rnn(rnn_cell, input_data,
initial_state=initial_state,
dtype=tf.float32)
```


```
# create a BasicRNNCell
rnn_cell = tf.compat.v1.nn.rnn_cell.BasicRNNCell(hidden_size)
# 'outputs' is a tensor of shape [batch_size, max_time,
cell_state_size]
# defining initial state
initial_state = rnn_cell.zero_state(batch_size, dtype=tf.float32)
# 'state' is a tensor of shape [batch_size, cell_state_size]
outputs, state = tf.compat.v1.nn.dynamic_rnn(rnn_cell, input_data,
initial_state=initial_state,
dtype=tf.float32)
```


Documentation

Creates a recurrent neural network specified by RNNCell cell. (deprecated)

Migrate to TF2

tf.compat.v1.nn.dynamic_rnn is not compatible with eager execution and tf.function. Please use tf.keras.layers.RNN instead for TF2 migration. Take LSTM as an example, you can instantiate a tf.keras.layers.RNN layer with tf.keras.layers.LSTMCell, or directly via tf.keras.layers.LSTM. Once the keras layer is created, you can get the output and states by calling the layer with input and states. Please refer to this

guide for more details about
Keras RNN. You can also find more details about the difference and comparison between Keras RNN and TF compat v1 rnn in this
document

Structural Mapping to Native TF2

Before:

After:

How to Map Arguments

Description

Performs fully dynamic unrolling of inputs.

Example:

Returns

If time_major == False (default), this will be a Tensor shaped:
[batch_size, max_time, cell.output_size].

If time_major == True, this will be a Tensor shaped:
[max_time, batch_size, cell.output_size].

Note, if cell.output_size is a (possibly nested) tuple of integers
or TensorShape objects, then outputs will be a tuple having the
same structure as cell.output_size, containing Tensors having shapes
corresponding to the shape data in cell.output_size.

Raises